

Module 04 Lab Guide

Simulating Spiking Neurons with Python

Houston City College • Spring 2026

Before You Begin

This guide walks you through building a spiking neuron simulation step by step.

You will NOT find complete solutions here — you will find explanations, starter code, hints, and tasks where you write the code yourself.

Answer all Knowledge Checks and Reflections in Markdown cells in your notebook.

Platform: Google Colab (colab.research.google.com)

File name: L04_FirstName_LastName_ITAI4374.ipynb

Part A: Setup and Your First LIF Neuron

Step 1: Set Up Your Notebook

Create a new Google Colab notebook and start with an import cell:

```
import numpy as np
import matplotlib.pyplot as plt

print('Setup complete!')
```

Run this cell (Shift+Enter). If you see “Setup complete!” you are ready to go.

Step 2: Understand the LIF Equation

The Leaky Integrate-and-Fire (LIF) neuron is the simplest model that captures spiking behavior. Remember the equation from the booklet:

```
# The LIF equation (discrete time version):
#
#   V[t+1] = V[t] + dt * ( -(V[t] - V_rest) / tau + I[t] / C )
#
# Where:
#   V       = membrane potential (voltage)
#   V_rest  = resting potential (what V returns to with no input)
#   tau     = membrane time constant (how fast voltage decays)
#   I       = input current
#   C       = membrane capacitance
#   dt      = time step size
```

Think of it like a leaky bucket: current pours in (I), but voltage constantly leaks back toward rest (the $-(V - V_{\text{rest}})/\tau$ term). If enough current accumulates to push V above a threshold, the neuron fires a spike and resets.



The “leaky” part is the key: without input, the voltage always decays back to rest. This is what makes it biologically realistic — real neurons leak ions through their membrane too.

Step 3: Define Your Neuron Parameters

Before writing the simulation, set up the parameters. Create a new code cell:

```
# ===== Simulation Parameters =====
dt = 0.001          # Time step (1 ms)
T = 1.0             # Total simulation time (1 second)
time = np.arange(0, T, dt) # Time array

# ===== Neuron Parameters =====
V_rest = -70.0      # Resting potential (mV)
V_thresh = -55.0    # Spike threshold (mV)
V_reset = -75.0     # Reset potential after spike (mV)
tau = 0.020        # Membrane time constant (20 ms)
C = 1.0            # Membrane capacitance (normalized)

print(f'Simulation: {T}s with {len(time)} time steps')
print(f'Threshold: {V_thresh} mV, Rest: {V_rest} mV')
```

Run this cell and verify the output makes sense.

KNOWLEDGE CHECK

Answer these in a Markdown cell below your code.

1. Why is V_{reset} (-75 mV) lower than V_{rest} (-70 mV)? What biological phenomenon does this represent?
2. The time constant τ is 20 ms. In plain English, what does this control?

Step 4: Build the LIF Simulation

Now write the core simulation loop. This is where you implement the LIF equation. Study the starter code below — you need to fill in the missing pieces.

YOUR TASK 1: Implement the LIF Neuron Loop

Complete the simulation loop. Read each TODO carefully and use the hints provided.

```
# ===== Input Current =====
# Constant current injection (we'll make this fancier later)
I_input = np.zeros(len(time))
I_input[200:800] = 15.0 # Inject 15 nA from t=200ms to t=800ms

# ===== Storage Arrays =====
V = np.zeros(len(time)) # Membrane potential over time
V[0] = V_rest           # Start at resting potential
spikes = []             # Store spike times

# ===== Simulation Loop =====
for t in range(len(time) - 1):

    # TODO 1: Compute the voltage change using the LIF equation
    #  $dV = dt * ( -(V[t] - V\_rest) / \tau + I\_input[t] / C )$ 
    # Hint: This is one line. Use the variables defined above.
    dV = _____ # YOUR CODE HERE

    # TODO 2: Update the membrane potential
    # Hint: V at the next time step = V at current step + dV
    V[t+1] = _____ # YOUR CODE HERE

    # TODO 3: Check if voltage crossed the threshold
    # If V[t+1] >= V_thresh:
    #   - Record the spike time (append time[t+1] to spikes list)
    #   - Reset V[t+1] to V_reset
    # Hint: This is an if statement with two lines inside
    # YOUR CODE HERE (2-3 lines)

print(f'Neuron fired {len(spikes)} spikes')
```

TIP

If your neuron fires 0 spikes, double-check your dV equation — make sure the signs are right.
If it fires thousands of spikes, make sure you are resetting V to V_reset (not V_rest) after each spike.

CHECKPOINT

Your neuron should fire roughly 10–25 spikes with these parameters. The exact number depends on your implementation. If you get a number in that range, you are on track.

Step 5: Visualize the Membrane Potential

Now plot what happened inside your neuron. This is where you will see the spikes.

YOUR TASK 2: Create the Membrane Potential Plot

Build a plot with two subplots: (1) membrane potential over time, and (2) input current over time.

```
fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(12, 6), sharex=True)

# TODO 4: Plot the membrane potential on ax1
# Hint: ax1.plot(time, V)
# Add a horizontal dashed line at V_thresh to show the threshold
# Hint: ax1.axhline(y=V_thresh, color='r', linestyle='--', label='Threshold')
# YOUR CODE HERE (2-3 lines)

ax1.set_ylabel('Membrane Potential (mV)')
ax1.set_title('LIF Neuron Simulation')
ax1.legend()

# TODO 5: Plot the input current on ax2
# YOUR CODE HERE (1-2 lines)

ax2.set_ylabel('Input Current (nA)')
ax2.set_xlabel('Time (s)')

plt.tight_layout()
plt.show()
```

✓ CHECKPOINT

You should see the voltage rising when current is on, spikes hitting the threshold, and resets. The bottom plot shows when current is injected.

✓ KNOWLEDGE CHECK

Answer these in a Markdown cell below your code.

1. Look at your membrane potential plot. What happens to V between spikes — does it go straight up, or does it curve? Why?
2. What happens to V after the current turns off at $t=800\text{ms}$? Why does it behave this way?

🧠 REFLECTION

Look at your membrane potential trace. In the booklet, you learned that biological neurons show similar integrate-and-fire behavior. What aspects of your plot look “realistic” compared to what you read about real neurons? What looks overly simplified?

Part B: Encoding Data as Spikes

Real brains don't receive numbers or pixel values — they receive spikes. To feed data into an SNN, we need to convert regular numbers into spike trains. The two main strategies are rate coding and temporal coding.

Step 6: Understand Rate Coding

In rate coding, higher values produce more spikes per second. Think of it like this: a bright pixel fires frequently, a dark pixel fires rarely.

The simplest implementation: at each time step, generate a random number. If the random number is less than the pixel intensity (scaled to 0–1), emit a spike.

YOUR TASK 3: Implement Rate Coding

Write a function that converts a 1D array of values (e.g., pixel intensities) into spike trains using rate coding.

```
def rate_encode(values, num_steps=100):
    """
    Convert an array of values (0 to 1) into spike trains.

    Args:
        values: 1D numpy array with values between 0 and 1
        num_steps: number of time steps to simulate

    Returns:
        spikes: 2D array of shape (num_steps, len(values))
                where 1 = spike, 0 = no spike
    """
    num_neurons = len(values)
    spikes = np.zeros((num_steps, num_neurons))

    for t in range(num_steps):
        # TODO 6: Generate random numbers and compare to values
        # For each neuron, if random < value, emit a spike (set to 1)
        # Hint: np.random.rand(num_neurons) gives random numbers
        # Hint: You can compare arrays: (random_array < values) gives True/False
        # Hint: Use .astype(float) to convert True/False to 1/0
        spikes[t] = _____ # YOUR CODE HERE
```

```
return spikes
```

Test your function with a simple example:

```
# Test with 5 values of increasing intensity
test_values = np.array([0.1, 0.3, 0.5, 0.7, 0.9])
test_spikes = rate_encode(test_values, num_steps=200)

# Count spikes per neuron
for i, v in enumerate(test_values):
    count = test_spikes[:, i].sum()
    print(f'Value {v:.1f} -> {count:.0f} spikes in 200 steps')
```

CHECKPOINT

Higher values should produce more spikes. Value 0.9 should have roughly 180 spikes, value 0.1 roughly 20. Your exact numbers will vary (it's random), but the trend should be clear.

Step 7: Understand Temporal Coding

In temporal coding, the information is in WHEN a spike happens, not how many. Higher values spike earlier; lower values spike later (or not at all).

Think of it as urgency: important (high-value) signals arrive first.

YOUR TASK 4: Implement Temporal Coding

Write a function that converts values into spike trains where the spike TIME encodes the value.

```
def temporal_encode(values, num_steps=100, threshold=0.05):
    """
    Convert values (0 to 1) into spike trains using time-to-first-spike.
    Higher values spike earlier.

    Args:
        values: 1D numpy array with values between 0 and 1
        num_steps: number of time steps
        threshold: minimum value to produce a spike
    Returns:
        spikes: 2D array (num_steps, len(values))
    """
```

```

num_neurons = len(values)
spikes = np.zeros((num_steps, num_neurons))

for i in range(num_neurons):
    if values[i] > threshold:
        # TODO 7: Calculate spike time
        # Higher values should spike at EARLIER time steps
        # Hint: spike_time = int((1 - values[i]) * (num_steps - 1))
        # This maps value=1.0 to time=0 and value=0.05 to time~95
        spike_time = _____ # YOUR CODE HERE
        spikes[spike_time, i] = 1

return spikes

```

☑ KNOWLEDGE CHECK

Answer these in a Markdown cell below your code.

1. In rate coding, what information is lost about the original signal? What is preserved?
2. In temporal coding, why might it be faster for a downstream neuron to make a decision compared to rate coding?
3. Can you think of a real-world example where timing matters more than frequency? (Hint: think about sound localization.)

Step 8: Visualize and Compare Encodings

Now apply both encoding methods to the same data and compare them visually using raster plots.

YOUR TASK 5: Create Raster Plots for Both Encodings

Load a simple test signal (a gradient from 0 to 1), encode it with both methods, and plot raster plots side by side.

```

# Create a test signal: 20 values from 0.05 to 1.0
signal = np.linspace(0.05, 1.0, 20)

# Encode with both methods
rate_spikes = rate_encode(signal, num_steps=200)
temp_spikes = temporal_encode(signal, num_steps=200)

# TODO 8: Create a figure with 2 subplots side by side
# Left: raster plot for rate coding
# Right: raster plot for temporal coding
#
# For a raster plot, you need to find where spikes occurred:
#     times, neurons = np.where(spikes_array == 1)

```



```
# ax.scatter(times, neurons, s=1, color='black')
#
# Label axes: x = 'Time Step', y = 'Neuron (by value)'
# Title each subplot appropriately
#
# YOUR CODE HERE (8-12 lines)
```

CHECKPOINT

Rate coding raster: scattered dots everywhere, denser at top (high-value neurons). Temporal coding raster: a diagonal line — high-value neurons spike on the left, low-value on the right.

REFLECTION

Compare your two raster plots. Which encoding would be better for a system that needs to react quickly? Which would be better for a system that needs precise measurements? The brain uses both strategies in different circuits — why do you think that is?

Part C: Connect a Mini Spiking Network

A single neuron is interesting, but brains are networks. Now you will connect multiple LIF neurons with synaptic weights to see how spikes propagate through a network.

The idea is simple: when neuron A fires a spike, it sends current to neuron B. The amount of current depends on the synaptic weight between them. Positive weights are excitatory (help B fire), negative weights are inhibitory (prevent B from firing).

Step 9: Design the Network

We will build a small network with 5 neurons:

- Neurons 0 and 1: Input neurons (receive external current)
- Neurons 2 and 3: Hidden neurons (receive input from neurons 0 and 1)
- Neuron 4: Output neuron (receives input from neurons 2 and 3)

YOUR TASK 6: Define the Network and Simulate

Create a weight matrix and run the simulation. When a neuron spikes, add weighted current to its downstream neurons.

```
# Network parameters
N = 5 # Number of neurons
T_net = 0.5 # Simulation time
time_net = np.arange(0, T_net, dt)
steps = len(time_net)

# Weight matrix: weights[i][j] = connection from neuron i to neuron j
# Positive = excitatory, Negative = inhibitory, 0 = no connection
weights = np.zeros((N, N))
weights[0, 2] = 12.0 # Neuron 0 -> 2 (excitatory)
weights[1, 3] = 12.0 # Neuron 1 -> 3 (excitatory)
weights[2, 4] = 10.0 # Neuron 2 -> 4 (excitatory)
weights[3, 4] = 10.0 # Neuron 3 -> 4 (excitatory)

# External input: only to neurons 0 and 1
I_ext = np.zeros((steps, N))
I_ext[100:400, 0] = 18.0 # Neuron 0 gets current
I_ext[150:350, 1] = 18.0 # Neuron 1 gets current (slightly offset)
```

```

# Storage
V_net = np.full((steps, N), V_rest) # All neurons start at rest
spike_log = [[] for _ in range(N)] # Spike times per neuron

# ===== Simulation Loop =====
for t in range(steps - 1):
    for n in range(N):

        # TODO 9: Compute total input current for neuron n
        # Total current = external input + synaptic input from other neurons
        # Synaptic input: if any neuron j spiked at time t,
        #   add weights[j, n] to the current
        # Hint: Check if time[t] is in spike_log[j] for each j
        #   OR use a simpler approach: track which neurons spiked
        #   at this time step with a separate array
        I_total = I_ext[t, n] # Start with external input
        # YOUR CODE HERE: add synaptic input from other neurons

        # TODO 10: Update membrane potential (same LIF equation as before)
        dV = _____ # YOUR CODE HERE
        V_net[t+1, n] = _____ # YOUR CODE HERE

        # TODO 11: Check for spike and reset (same as before)
        # If spiked, record time_net[t+1] in spike_log[n]
        # YOUR CODE HERE

# Print results
for n in range(N):
    print(f'Neuron {n}: {len(spike_log[n])} spikes')

```

TIP

The synaptic input is the tricky part. One approach: at each time step, create a “just_spiked” boolean array. Then for neuron n, sum weights[j, n] for all j where just_spiked[j] is True. Alternatively, you can simplify by checking if V_net[t, j] was just reset (meaning it spiked).

CHECKPOINT

Input neurons (0, 1) should fire the most. Hidden neurons (2, 3) should fire next. Output neuron (4) should fire last and fewest. If neuron 4 has zero spikes, try increasing the weights slightly.

Step 10: Visualize Network Activity

YOUR TASK 7: Create a Network Raster Plot

Plot a raster plot showing all 5 neurons' spike times, plus a subplot showing the membrane potential of the output neuron (neuron 4).

```
fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(12, 6), sharex=True)

# TODO 12: Raster plot of all neurons
# For each neuron n, plot its spike times as dots at y = n
# Use different colors for input (blue), hidden (green), output (red)
# Hint: ax1.scatter(spike_log[n], [n]*len(spike_log[n]), ...)
#
# YOUR CODE HERE

ax1.set_ylabel('Neuron')
ax1.set_title('Network Spike Raster')
ax1.set_yticks(range(N))
ax1.set_yticklabels(['Input 0', 'Input 1', 'Hidden 2', 'Hidden 3', 'Output 4'])

# TODO 13: Plot output neuron membrane potential
# Hint: ax2.plot(time_net, V_net[:, 4])
# YOUR CODE HERE

ax2.set_ylabel('V (mV)')
ax2.set_xlabel('Time (s)')
ax2.set_title('Output Neuron (4) Membrane Potential')

plt.tight_layout()
plt.show()
```

☒ KNOWLEDGE CHECK

Answer these in a Markdown cell below your code.

1. Does the output neuron fire at the same rate as the input neurons? Why or why not?
2. What role do the synaptic weights play? What would happen if you doubled them? Halved them?
3. In the brain, not all connections are excitatory. What might happen if you made weights[3, 4] negative (inhibitory)?

Part D: Experiments

Science is about changing things and observing what happens. Run at least 3 of the experiments below. For each one, change the parameter, re-run your simulation, and document what you observe.

WARNING

For each experiment: create a new code cell, copy and modify the relevant code, run it, and write your observations in a Markdown cell below. Do NOT overwrite your original working code.

Experiment Menu (choose at least 3)

Experiment 1: Threshold Sensitivity

Go back to your single LIF neuron (Part A). Change V_{thresh} from -55 to -50, then to -60. How does the firing rate change? Plot both and compare.

Experiment 2: Time Constant Effects

Change τ from 20 ms to 10 ms, then 50 ms. How does this affect the shape of the membrane potential curve between spikes?

Experiment 3: Varying Input Current

Instead of constant current, try a gradually increasing current: $I_{\text{input}} = \text{np.linspace}(0, 25, \text{len}(\text{time}))$. How does the firing pattern change over time?

Experiment 4: Noisy Input

Add random noise to your constant current: $I_{\text{noisy}} = 15.0 + \text{np.random.randn}(\text{len}(\text{time})) * 5.0$. How does noise affect spike timing? Is the pattern regular or irregular?

Experiment 5: Network Weight Changes

In your mini SNN, try making one connection inhibitory (e.g., $\text{weights}[3, 4] = -8.0$). How does this change the output neuron's behavior?

Experiment 6: Firing Rate Curve

Sweep input current from 0 to 30 in steps of 1. For each current level, count how many spikes your LIF neuron fires. Plot current (x-axis) vs. spike count (y-axis). This is called an f-I curve.

For Each Experiment, Write:

1. What parameter did you change, and to what values?
2. What did you expect to happen?
3. What actually happened? (Include the plot.)
4. Why does this make sense given the LIF equation?

Part E: Final Reflection

Create a Markdown cell with the heading “## Final Reflection” and answer the following questions. Write thoughtfully — these connect your hands-on experience to the bigger picture from the booklet.

REFLECTION

1. How does changing the membrane time constant (τ) affect firing behavior? What biological property does this represent? (Think about different types of neurons in the brain.)

REFLECTION

2. Compare your rate coding and temporal coding results. Which encoding preserved more information about the original signal? Why might the brain use both strategies in different circuits?

REFLECTION

3. In your mini SNN, what happened when spikes propagated through layers? How does this relate to how information flows through real neural circuits?

REFLECTION

4. The LIF model is a simplification. What does it capture well about real neurons? What important biological features does it miss? (Think: dendrites, neurotransmitters, morphology, adaptation.)

REFLECTION

5. After building these simulations, what is one thing about biological neural computation that surprised you or changed how you think about artificial intelligence?

 **You're Done!**

Make sure your notebook contains: all code cells with outputs visible, all Knowledge Check answers, all Experiment observations, and the Final Reflection.

Save and download: File → Download → Download .ipynb

File name: L04_FirstName_LastName_ITAI4374.ipynb

Submit your single notebook to Canvas.